Reviewer Pack — Erdős-Rado Sunflower Petals Lower-Bound Lifted Log-Rank for …

Entry f1f0ed613240 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 18:31:51 UTC

Erdős-Rado Sunflower Petals Lower-Bound Lifted Log-Rank for IND_2

Entry ID: `f1f0ed613240` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-28 18:31:51 UTC

1. Conjecture

Field A (mathematical branch): Extremal set theory: the Erdős-Rado sunflower lemma — maximum number r(F) of sets in a family F admitting a common core Y with pairwise petal-disjoint complements S_i Y (Erdős-Rado 1960; Alweiss-Lovett-Wu-Zhang 2020 quasi-polynomial improvement; Rao 2020 simplification). Distinct from Razborov's monotone-circuit approximation method (which uses sunflowers on minterms in a min-AND-OR closure context), and rarely deployed as a quantitative invariant against deterministic communication complexity of lifted functions. **Field B** (complexity object): Deterministic two-party communication complexity of the indexing-lifted Boolean function f∘IND_2 for f:{0,1}^k → {0,1}, accessed via the real log-rank lower bound D^cc_log(f∘IND_2) := ⌈log₂ rk_R(M_{f∘IND_2})⌉ on the 2^k × 4^k lifted communication matrix M_{f∘IND_2}[x,y] = f(y_1[x_1], …, y_k[x_k]). This is the canonical Raz-McKenzie / Göös-Pitassi-Watson lifting target object at the smallest nontrivial gadget arity b=2.

Statement:

> For every Boolean function f:{0,1}^k → {0,1} with k ∈ {2,3,4}, let C_min(f) be the family of minimal 1-certificates of f (minimal partial assignments α ⊂ [k]×{0,1} with f|_α ≡ 1) and let r(f) be the maximum sunflower size in C_min(f), i.e. the largest m such that there exist α_1, …, α_m ∈ C_min(f) and a core Y ⊆ ⋃*i α_i with the petals α_i Y pairwise disjoint as literal sets. Then ⌈log₂ r(f)⌉ ≤ D^cc_log(f∘IND_2). Any f:{0,1}^k → {0,1} (k ≤ 4) with ⌈log₂ r(f)⌉ > ⌈log₂ rk_R(M*{f∘IND_2})⌉ refutes the conjecture.

Rationale (proposer's reasoning):

> In the Raz-McKenzie / GPW lifting picture, IND_2 hands Bob a two-element 'choice' per coordinate while Alice picks the index, so a sunflower of m 1-certs with common core Y and disjoint petals should embed m linearly independent rank-1 patterns into M_{f∘IND_2}: the core fixes a shared structural part of the lifted matrix, while the disjoint petals act on disjoint y-coordinates and therefore should be R-linearly independent in the row span. Translating Erdős-Rado's structure-vs-randomness dichotomy into a log-rank witness would expose a combinatorial source of communication hardness orthogonal to spectral / discrepancy bounds. This bridge is also natural-proofs-safe: r(f) is not a 'large dense useful' property in the Razborov-Rudich sense (it is bounded by 2^k for k≤4 and tied to a specific algebraic invariant rk_R, not to arbitrary circuit size).

Taxonomy category: `LIFTING` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `0c873af1ea057046`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across all enumerated f for k∈{2,3} ($2^{4+2}8$=272 functions) and all sampled/symmetric f for k=4, the conjecture is SUPPORTED iff $\lfloor \log_2 r(f) \rfloor \leq \lceil \log_2 rk\_R(M\_\{f\circ IND\_2\}) \rceil$ holds for 100% of tested f (single deterministic run; rank computed with numpy default tol).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `sunflower lemma communication complexity lifting indexing gadget` - `Erdős-Rado sunflower log-rank lower bound lifted Boolean function` - `minimal certificates sunflower deterministic communication Raz-McKenzie IND gadget`

Top relevant hits considered: - [http://arxiv.org/abs/1904.13056v4] Query-to-Communication Lifting Using Low-Discrepancy Gadgets - [http://arxiv.org/abs/2211.17211v2] On Disperser/Lifting Properties of the Index and Inner-Product Functions - [http://arxiv.org/abs/1809.10318v3] Improved Bound on Sets Including No Sunflower with Three Petals - [http://arxiv.org/abs/2505.03671v2] The Erdős-Rado Sunflower Problem for Vector Spaces - [http://arxiv.org/abs/2511.07739v3] A Lower Bound for the Fourier Entropy of Boolean Functions on the Biased Hypercube - [http://arxiv.org/abs/2012.03883v2] Monotone Circuit Lower Bounds from Robust Sunflowers - [http://arxiv.org/abs/astro-ph/0003162v3] GADGET: A code for collisionless and gasdynamical cosmological simulations - [http://arxiv.org/abs/1506.08263v1] Schubert decompositions for ind-varieties of generalized flags

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
import json

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def matrix_rank(M):
```

```
    m, n = len(M), len(M[0])
    rank = 0
    for i in range(m):
        if any(M[i][j] != 0 for j in range(n)):
            rank += 1
            for j in range(n):
                M[i][j] /= M[i][j]
            for k in range(m):
                if k != i and any(M[k][j] != 0 for j in range(n)):
                    for j in range(n):
                        M[k][j] -= M[k][i] * M[i][j]
    return rank

def gaussian_elimination(A, b):
    m, n = len(A), len(A[0])
    augmented = [A[i] + [b[i]] for i in range(m)]
    for i in range(n):
        max_row = i
        for j in range(i+1, m):
            if abs(augmented[j][i]) > abs(augmented[max_row][i]):
                max_row = j
        augmented[i], augmented[max_row] = augmented[max_row], augmented[i]
        factor = 1 / augmented[i][i]
        for j in range(n + 1):
            augmented[i][j] *= factor
        for j in range(m):
            if j != i:
                factor = augmented[j][i]
                for k in range(n + 1):
                    augmented[j][k] -= factor * augmented[i][k]
    return [row[-1] for row in augmented]

def is_linearly_independent(vectors):
    m, n = len(vectors), len(vectors[0])
    A = [vectors[i] + [1] for i in range(m)]
    return matrix_rank(A) == n

def min_1_certificates(f, k):
    certificates = []
    for alpha in itertools.product([0, 1], repeat=k):
        if f(alpha) == 1:
            certificates.append(alpha)
    return certificates

def sunflower_size(certificates):
    max_size = 0
    for i in range(len(certificates)):
        core = set()
        petals = []
        for j in range(i+1, len(certificates)):
            common_part = set(zip(certificates[i], certificates[j]))
            if not core.intersection(common_part):
                core.update(common_part)
                petals.append(set(certificates[j]) - core)
        max_size = max(max_size, len(core) + len(petals))
    return max_size
```

```
    def f(alpha):
        return alpha[0] ^ alpha[1]

    k_values = [2, 3, 4]
    results = []

    for k in k_values:
        if k == 4:
            functions = [f for _ in range(2000)] + [lambda x: x[0] == x[1]] + [lambda x: x[0] != x
        else:
            functions = [lambda alpha, f=f: all(f(alpha[:i] + (b,) + alpha[i+1:]) == 1 for b in [0

        for f in functions:
            certificates = min_1_certificates(f, k)
            r_f = sunflower_size(certificates)
            M = [[f(alpha[:i] + (j,) + alpha[i+1:]) for j in range(2**k)] for i in range(k)]
            rk_M = matrix_rank(M)
            results.append((r_f, rk_M))

    metric_name = "log_rk_ratio"
    metric_value = sum(math.log2(r) - math.ceil(math.log2(rk)) for r, rk in results) / len(results
    instances_tested = len(results)
    conjecture_holds = all(math.log2(r) <= math.ceil(math.log2(rk)) for r, rk in results)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(json.dumps({"SEED": seed, **result}))

    results = [run_trial(seed) for seed in seeds]
    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / le
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a37580e9.py", line 116, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a37580e9.py", line 92, in run_trial
    certificates = min_1_certificates(f, k)
                   ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a37580e9.py", line 62, in min_1_certifi
    if f(alpha) == 1:
       ^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a37580e9.py", line 89, in <lambda>
    functions = [lambda alpha, f=f: all(f(alpha[:i] + (b,) + alpha[i+1:]) == 1 for b in [0, 1]) for alpha
                                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a37580e9.py", line 89, in <genexpr>
    functions = [lambda alpha, f=f: all(f(alpha[:i] + (b,) + alpha[i+1:]) == 1 for b in [0, 1]) for alpha
                                                    ^
NameError: name 'i' is not defined. Did you mean: 'id'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with a NameError (‘name i is not defined’) in the lambda construction before any conjecture data was produced, so neither the support nor the falsification condition can be evaluated. | next: Fix the lambda scoping bug (e.g., bind i as a default argument in the generator) and rerun the full enumeration for k∈{2,3,4}.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	320617
2	preregistration	claude_max	opus	0	0	6419
3	novelty	claude_max	opus	0	0	3573
4	novelty	claude_max	opus	0	0	8703
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12948
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14124
7	judge	claude_max	opus	0	0	4552

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 370937 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/f1f0ed613240.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f1f0ed613240.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/f1f0ed613240.tar.gz` (if generated)